

CESE 4025 - Real-Time Systems: Assignment B, Group: 02

Zubair Al-Zubi (5342007), Mustafa Yücesan (5655544)

January 18, 2025

GitLab commit hash for parts 1,2 & 3: ff6b940a84be6387e82e24df6312c394263937c7

GitLab commit hash for part 4: 8f0073bbebcc3bb2782fabbece64f80beba98bc7

Report contains 8 of maximum 10 pages.

1 Introduction

For this assignment, a faulty synthesizer was provided, and the aim was to analyze its problems and implement solutions such that it can procure sound clearly. Initially, the average and worst case execution times of important tasks were analyzed and measured to understand their performance requirements. After that, tasks were separated into threads based on their priorities to improve efficiency, and potential race conditions were identified. Double buffering was also implemented to eliminate audio glitches. Lastly, overload detection and handling was implemented to manage pressing multiple keys at the same time, and interrupt routines were used instead of polling to improve responsiveness.

2 Part 1 - System specifications

In Part 1, the average (AECT) and worst case execution time (WCET) will be measured for Task 3 (sound synthesis) and 4 (audio output to the amplifier) using the logic analyzer software **Saleae Logic 2** while the **SW_OSC1** switch is in the "Up" position and only one note is playing.

Table 1: Task to LED mapping and corresponding logic analyzer channels

Task	LED	Logic Analyzer Channel
Task 1: Peripheral Input	Green (LD4)	CH0
Task 2: Keyboard Input	Orange (LD3)	CH1
Task 3: Synth Sound	Red (LD5)	CH2
Task 4: Audio Block Write	Blue (LD6)	CH3

From the setup in **Table 1**:

- **Channel 2** represents **Task 3**,
 - **Channel 3** represents **Task 4**.
- Using Logic 2, the measured AECT and WCET are shown in **Table 2**.

Statement on the use of generative AI* (tick what applies, multiple options are possible):

When doing this assignment, I/we:

0 Did not use generative AI.

0 Used generative AI to generate (parts of) the code. Where applicable, I/we have explained this in a comment.

0 Used generative AI to generate (parts of) the report. Where applicable, I/we have explained this in a footnote.

1 Used generative AI to lint/format the code and/or improve the grammar, style, spelling of the report.

Table 2: Measured AECT and WCET for Tasks 3 and 4

Task	Average Execution Time (AECT) [ms]	Worst-Case Execution Time (WCET) [ms]
Task 3 (CH2)	6.333	6.364
Task 4 (CH3)	42.964	42.992

○ For Tasks 1 and 2:

■ The use cases of tasks 1 and 2:

- **Task 1 (Peripheral Input):** The average execution time occurs when a single peripheral input (e.g., a switch) is triggered once. The worst case execution time occurs when multiple peripherals are triggered simultaneously or in rapid succession.
- **Task 2 (Keyboard Input):** The AECT for this task occurs with a single key press, while the WCET is measured when multiple keys are pressed simultaneously or rapidly, like during key mashing.

■ The measured AECT and WCET for Tasks 1 and 2 are summarized in [Table 3](#). This was done by toggling multiple switches and changing rotary encoders at the same time for Task 1, and by pressing multiple keyboard keys at the same time for Task 2.

Table 3: Measured AECT and WCET for Tasks 1 and 2 when recreating the use cases

Task	AECT	WCET
Task 1 (Peripheral Input, CH0)	818 μ s	4.246 ms
Task 2 (Keyboard Input, CH1)	2 μ s	10 μ s

■ The Processor Utilization (PU) for Tasks 1 and 2 was calculated as follows:

$$PU = \frac{\text{Execution Time}}{\text{Task Period}} \times 100\%,$$

and the recorded values are listed in [Table 4](#).

Table 4: Processor Utilization (PU) for Tasks 1 and 2

Task	PU (AECT)	PU (WCET)
Task 1 (Peripheral Input)	1.63%	8.47%
Task 2 (Keyboard Input)	0.004%	0.004%

○ Task 3's PU can be calculated based on the remaining available processor time after higher-priority tasks (Task 1 and Task 2) are accounted for.

$$PU_{\text{Task 3}} = 100\% - PU_{\text{Task 1}} - PU_{\text{Task 2}} = 100\% - 1.63\% - 0.004\% = 98.4\%.$$

$$PU_{\text{Task 3, WCET}} = 100\% - PU_{\text{Task 1, WCET}} - PU_{\text{Task 2, WCET}} = 100\% - 8.47\% - 0.004\% = 91.5\%.$$

Thus, the remaining PU assigned to Task 3 would be between 91.5% and 98.4%.

3 Part 2 - Superloop split up

3.1 Design specifications

3.1.1 Upper and lower bounds of task periods

Task 1 reads input from peripherals and updates their state. This is the most important bit, because tasks 2-4 all depend on this, if they ignore this they will cause the wrong sound to be produced.

THE WCET of task 1 is approximately 4ms. This means around 4ms should usually be sufficient to complete the task. The AECT of this task is very low. The minimum period we define to be 10 ms, so we ensure it executes.

While one could argue a lower value could be defined, a counterargument would be that it doesn't need to be that fast and a lower bound would put strain on the system. This is sufficient for peripherals. As an upper bound, we choose 20, as we do not want it to be too slow.

It is also important to take into account the rotary encoder here, because the speed it is turned can vary significantly, whereas with a button you just press it. We choose 15ms as the period for task 1 as a halfway measure. With initially testing, the switches react properly, and the rotary adjusts correctly. Previously, other periods were tested but we found the values don't change smoothly, depending on how fast the rotary encoder is turned.

Task 2 processes key presses send via serial. We want this to be quick, but not unnecessarily quick which might result in unnecessary overload. However, in table 3 we can see detecting keypresses is extremely fast, namely 2 μ s for WCET. We deem a lower bound of 1 ms and an upper bound of 2 ms sufficient. As you can see, the interval is very low. Due to the WCET being so fast, we can just go with a period of 1ms, which is the lower bound. But we surely don't want the upper bound to be much more than 1 ms.

Task 3 is about the synthesis generation. It processes the audio data to generate sound using the synthesizer. Since it is about audio, there is likely a sampling rate, and the system response will be bound by that. The standard audio sampling rate is 44.1 kHz, which typically requires 1 ms to 10 ms, so we took 1 ms as a lower bound. We assumed that after 10 ms, a delay might be noticeable. We chose 5 ms to balance things out. The period here is estimated in theory, but when measures later the WCET and AECT were both around 6.5. We hereby adjusted the lower bound to 7, and the upper bound to 20, figuring the provided sample code is not the most optimal. We chose a period of 12 for this task in the end.

Please do note that in a later exercise interrupt were implemented. A requirement was that after an overload happened, the red led (D7) had to be turned on and off. However, in order to detect the overload an ISR was used. Within an ISR, however, there is no blocking, and there needed to be some delay between turning the led on and off, otherwise it never was visually on. This was fixed by adding a small sleep in the synthesiser thread at the beginning. There is also one at the end. Together they are approximately 12ms.

Task 4 writes the processed audio block to the audio driver for playback. Its period should align with the audio buffer requirements and be sufficient to handle the task's WCET. Based on Table 2, the WCET is 42.992 ms, so the lower bound is set to 45 ms, ensuring the task has enough time to complete. The upper bound is set to 60 ms, providing a reasonable margin for flexibility. We choose 50ms as the period.

The chosen values are summarized in [Table 5](#).

Table 5: Lower and upper period bounds, and chosen periods for each task.

Task	Lower Bound [ms]	Upper Bound [ms]	Chosen Period [ms]
Task 1: Peripheral Checking	10	20	15
Task 2: Keyboard Input	1	2	1
Task 3: Sound Synthesis	7	20	12
Task 4: Audio Output	45	60	50

3.1.2 Separating the superloop into threads

Because task 3 and task 4 are closely related, we will put those in a single thread.

Task 1 and 2 will have their own separate threads. As task 2, 3, 4 all depend on the configuration, task 1 will be the most important task. However, this isn't necessarily an event that we expect to happen periodically, rather, we just want to know if there is a configuration change. After that, task 2 is the most important one, because we need to know the key presses which will decide which sound will be produced. Lastly, we will produce and play the sounds, in a single thread because we want the synthesis and writing to audio buffer to align. Thus, the following initial thread division has been made:

- Thread 1: peripheral checking.
- Thread 2: Keyboard input.
- Thread 3: Sound synthesis and audio output.

Note that, after progressing through the assignment we made the decision to do it in four threads instead. This will be explained in part 3.

3.1.3 Priorities for each thread and role of thread executing main()

Priorities for each thread Priorities are important to ensure that tasks with more critical timing requirements run first, reducing delays and preventing important tasks from missing their deadlines. The reasoning behind each priority level is explained below and summarized in Table 6.

- **Task 4 (Audio Output):** This task was assigned the highest and non-interruptible priority (-1) to prevent audio drops and to ensure a continuous, uninterrupted sound output. Any delay in this task could result in noticeable distortions in the audio. Notice that writing to audio amplifier has a negative priority, which basically means it is a non-interruptible thread. It cannot be interrupted by higher-priority tasks.
- **Task 1 (Peripheral Input):** Peripheral inputs must be handled quickly to ensure that user actions are reflected directly in the synthesizer's behavior. Missing peripheral inputs can cause incorrect sound configurations.
- **Task 2 (Keyboard Input):** The keyboard input task was assigned a slightly lower priority than peripheral input, but it must still execute with minimal latency to ensure a responsive user experience.
- **Task 3 (Sound Synthesis):** Sound synthesis is essential for generating audio blocks but can it tolerate slightly lower priority compared to input tasks.

Table 6: Chosen priorities for tasks 1–4.

Thread / Task	Priority	Justification
Task 4 (Audio Output)	-1 (Highest, Non-interruptible)	Timely buffer output to the DAC prevents dropouts, but input tasks remain top-priority.
Task 1 (Peripheral Input)	1 (Highest)	Quick user actions require fast handling.
Task 2 (Keyboard Input)	2	Key presses must be detected with minimal latency for good user experience.
Task 3 (Sound Synthesis)	3	Updates must occur regularly, but can be slightly lower than immediate inputs.

Role of main():

- **Initialization:** main() configures hardware, initializes data structures, and spawns the real-time threads.
- **Background / idle function:** After setup, main() can remain at a low (idle) priority to perform non-time-critical activities or simply remain idle, allowing higher-priority threads to preempt it.

3.2 Verification

The logic analyzer output is shown in Figure 1.

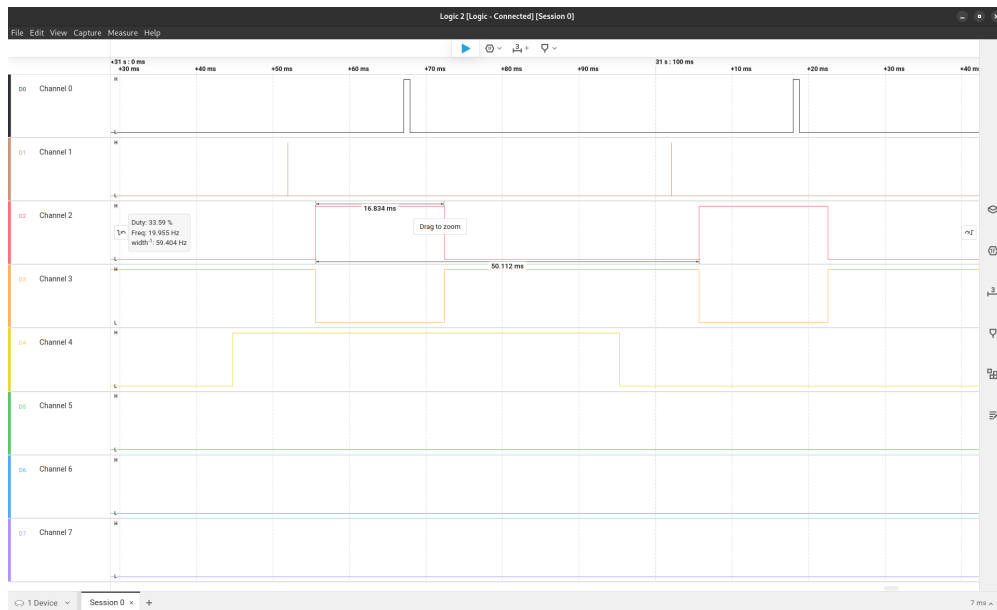


Figure 1: Logic analyzer output

Although "Task 3: Sound Synthesis" was initially set for 5 ms, it appears as 50 ms in Figure 1. This is due to a design choice to allow the LED to be visible while blinking; the sound is still being played as expected.

3.3 Race conditions

3.3.1 Identified race conditions

The following race conditions have been identified:

- LED: the functions `set_led()` and `reset_led()` are called in multiple threads. If these functions manipulate shared hardware resources or global variables, they could lead to race conditions.
- Keyboard state: the `check_keyboard()` function modifies the keys array, which is a shared resource. If other parts of the code (or other threads) access or modify this array, it could lead to race conditions.
- Semaphore usage: the semaphore `make_write_synth` is used to synchronize access to the `make_write_synth_thread`. If other threads also interact with this semaphore, it could lead to race conditions.

3.3.2 Race conditions analysis

The analysis of the identified race conditions is summarized in Table 7.

Table 7: Analysis of race conditions in the code

Race Condition	Tolerable/Needs Fix	Threads Involved	Triggering Operations	Impact and Fix Design
LED Manipulation	Tolerable	<code>peripheral.update_thread</code> , <code>check_keyboard_thread</code> , <code>make_write_synth_thread</code>	Concurrent calls to <code>set_led()</code> and <code>reset_led()</code>	Minimal impact on performance. Used for status indication. No fix required.
Key State Management	Needs Fix	<code>check_keyboard_thread</code>	Concurrent access to keys array	Can lead to incorrect key states. Fix: Use mutex to protect keys array.
Semaphore Usage	Needs Fix	<code>make_write_synth_thread</code>	Incorrect semaphore operations	Can cause missed or duplicate synth operations.

3.4 Double buffering

3.4.1 Problem description

Currently, the code uses a single buffer, which is used by both the sound generator and data transfer to the audio amplifier. These two tasks were placed sequentially in a single thread, which mitigates some of the negative effects of using a single buffer. Although, not completely. The presence of a single buffer is heard by a clipping background noise, which reduced in frequency when the application was threaded. This is likely due to each thread having their own corresponding period.

If both tasks were in their own threads, it might be the case that writing the audio has finished, but there is still sound generation being done. This will result in unwanted sounds, as you might be overwriting the data that has not played yet (with new sound generation). This makes sense, as threads can run at the same time. However, at this moment we have both synthesis and writing to driver in a single thread, yet the single buffer still causes an issue. This is much less intuitive, as the code is sequential.

It was expected that the synthesis is written to buffer, and when that is done it is written to driver. Only when that is done it will start processing the next audio frame. It turns out, this is not the case. Whenever the program wants to write to the driver, it sends an instruction over i2s using `i2s_write()`. This function puts the data to be sent on a queue. It takes ownership of the provided memory block, and will only release it when it is done. Thing is, after the instruction is sent successfully, the thread will finish. When it restarts again and the synthesiser writes to the buffer, it might be that it was still being read from in the hardware. This happens because of varying read and write speeds. As a solution, we implement double buffering.

3.4.2 Implementation description

Initially, we tried to set up our own memory slab, but later found the audio files already provide it. The number of memory blocks was changed to two from 1, such that we can make a second buffer. Then we use the provided `allocBlock()` function to create the buffers. Then we take one of the buffers and set it a variable pointer active, we also make a pointer for the inactive buffer.

Then we call the synthesis and write to driver functions on the active buffer, and then set the currently active buffer to inactive and vice versa. This ensures that the synthesis of the next audio frame does not touch the buffer that is currently being written out. Using the original audio driver's configuration and setting the exact number of blocks ensures memory allocation stays within the slab's size and avoids unnecessary allocation. Due to the nature of the `i2s_write()` function, it releases the memory block when it is done.

4 Part 3 - Overload

In this part, we implement overload detection and handling. This is mostly about the synthesizer task, as when you press multiple buttons at the same time it tries to synthesise multiple sounds at once. Now, for part 2, we had indicated that we kept the synthesis and writing to audio buffer in a single thread because we want the audio buffer to align. We thought that this would diminish the need for a double buffer but ended up learning that was not the case, as a hardware level process continued reading from the buffer while the thread was writing it's next audio frame to it. Now, we face a consequence of a decision we had made earlier. As the overload is mostly related to the synthesiser task, it is counterproductive to have the write to audio amplifier in the exact same thread.

Due to this, the synthesis and write to audio amplifier are separated in their own threads, as we need to measure overload of the synthesis only. By trial and error, we found that the upper bound is around 25ms. When it approaches that we call it an overload. With this upper bound a few keys at the same time have to be pressed for it to occur. It can be adjusted a bit down to allow for fewer keys, but an upper bound of 25ms appeared to work better. We used a timer for better precision.

When the timer detects the upper bound hits, it indicates an overload by going to its callback function. In the callback function, it aborts the synthesiser thread from running. In essence, we handle the overload by cancelling the current audio frame and starting the new one. The abort results in its resources being freed, so that nothing from the previous audio frame remains in its stack area. We also manually reset the audio buffers that are used in the double buffering. From the audio.cpp file, we calculated the size to be approximately 4410 bytes, and so

that amount will be set to 0's. The red LED is enabled to indicate an overload and then the synthesiser thread is restarted. When the red led was disabled in the same thread the LED was never shown to be on, probably due to it being too fast. Putting any amount of sleep before it didn't help either. We found this strange, but we later found that the timer callback runs in an interrupt context, and ISR is non-blocking, so putting a small sleep between turning the led on and off didn't work either. Instead, we turned the led off in the thread we recreated and this worked fine.

We also readjusted the priority of the writing to audio amplifier to -1, cause we figured that we never wanted this to be interrupted. It used to have the lowest priority before.

5 Part 4 - Interrupts

5.1 Switches

In this part, we implemented interrupts for a peripheral update functionality. Specifically, for the switches. Instead of polling to check for a change, an interrupt service routine (ISR) would be run when a switch changed state. The pin configurations of the switches were retrieved from the peripherals.cpp file. Note, that we code everything in the main, as we preferred less organisational complexity. Implementing interrupts also introduced the problem of bounce, which is basically what happens the contacts inside a switch fail to close smoothly. Rather, the internal components bounce of each other at that moment. This results in more than one signal being generated, where we only want one. If this is not fixed, the ISR would be initiated more than once each time.

First, let's describe how interrupts are implemented. Zephyr offers some functions to set up interrupts.

```
gpio_pin_interrupt_configure_dt(&sw_osc_dn, GPIO_INT_TRIG_LOW);
```

This function configures the pin interrupts for the switches, for both the up and down position. Before the interruption configuration, a normal pin configuration must be done. However, this is already done by the internal code provided. `GPIO_INT_TRIG_LOW` causes the switch to be detected.

```
static struct gpio_callback switch_data;
```

The `gpio_callback` structure is used to register a callback.

The `gpio_init_callback(switch_data, switch_switched, pin_mask)` function is used to initialise the structure. The pin mask contains all the pins that should trigger the callback. The `switch_switched` is the interrupt service routine that will be called when the switches switch. `gpio_add_callback(sw_osc_dn.port, &switch_data);` is used to add an application callback, which we do for every up and down position of every switch. To summarise, `gpio_init_callback()` initialises the callback structure with a pointer to the ISR function and a pin mask that specifies the pins of interest. It registers the initialised callback with the specified port, which is a pointer to the device structure for the driver instance, which allows it to actually communicate with the hardware.

Using the `gpio_callback_handler_t` function signature, the ISR function `switch_switched` was defined, which finalised setting up the interruptions for the switches. When this was initially set this up, there was a strange problem. When the switch was touched, without actually moving it, the capacitance resulted in the ISR being called, multiple times and quickly. This ended up being due to the charger being attached to the laptop. The hardware apparently wasn't adjusted to that. Initially, we thought it was bounce, but that shouldn't have not been so much repeated signals, and should only happen when switching. In the end, we just continued on battery.

After this, we noticed there was a bit of bounce after every time we switched. There are a special kind of threads to be used by ISR's, called work threads. Work threads are used by ISR to offload low urgent tasks so the ISR can remain non-blocking. It will yield every time an item is done. This we will use to fix the bouncing.

`static K_WORK_DELAYABLE_DEFINE(debounce_work, debounce_thread);` is used to create a work item called `debounce_work`, which will be takes care of by the `debounce_thread`. In `peripherals.h`, a `ThreePosSwitch` array is defined, which is initialised within `peripherals.cpp`. The update method of the `ThreePosSwitch` class is called on every switch that is in the array, and that retrieves the current configuration of all the switches and updates accordingly. It does this each time the ISR is run, which offloads the task to the work thread. Now, one might ask, what did this do about debouncing? Nothing as of yet, but in the ISR the work item is started as follows: `k_work_reschedule(&debounce_work, K_MSEC(15));`. The `K_MSEC(15)` tells it to submit (yield) the work item after a brief delay of 15ms. With bouncing, due to switch failing to close smoothly, it generates extra signals. This is fixed by simple waiting before submitting the work item after it has been called. The workthread, as defined by

the macro, is cooperative so before submitting no other threads can interrupt it. Meaning, it ignores the bounce. Note that a work thread can still specify it's priority, although not in the way we defined it. This way is more suitable for our task.

6 Conclusion

In conclusion, the initial faulty synthesizer was thoroughly analyzed and improved by optimizing task execution, addressing race conditions and implementing double buffering for smooth audio playback. Furthermore, overload detection and interrupt routines significantly improved the reliability and responsiveness of the system.

References

- [1] *Course info - Real-time Systems*. <https://cese.pages.ewi.tudelft.nl/real-time-systems/>.